Understanding AVR Timers and PWM

Timers

MikroDuino

Amer Iqbal Qureshi | Mikrotronics Pakistan

Timers: Teaching the Microcontroller About Time

Before we begin programming timers, let us first answer an important question:

Why do computers need timers at all?

The answer becomes obvious when we think about an everyday situation.

Imagine that you are baking a cake. The recipe says that it must remain in the oven for **exactly 30 minutes**. Once the cake is inside, would you stand in front of the oven staring at your watch and counting every passing second?

"One...
Two...
Three...
Four..."

Probably not.

That would be a terrible use of your time. Instead, you simply set a kitchen timer and continue doing other things. You might prepare the icing, wash the dishes, or read a book. When the thirty minutes have elapsed, the timer rings, reminding you that the cake is ready.

The timer quietly keeps track of time while you remain free to concentrate on more important work. A microcontroller faces exactly the same challenge.

At any given moment, the CPU may be busy performing dozens of different tasks. It may be reading temperature sensors, updating an LCD display, blinking LEDs, receiving serial data from another device, controlling motors, communicating with sensors over SPI or I²C, or processing user input from push buttons. All of these activities demand the CPU's attention.

Now imagine asking the CPU to keep track of time simply by counting every clock pulse generated by the processor's oscillator.

At a clock frequency of **16 MHz**, the processor receives **16 million clock pulses every second**. If the CPU had to count each of these pulses by itself, it would spend nearly all of its time watching the clock instead of performing useful work. There would be little time left to execute the actual program.

Fortunately, the designers of the AVR microcontroller solved this problem by including dedicated **hardware timers** inside the chip.

A hardware timer is a small digital circuit that works independently of the CPU. Once configured, it begins counting clock pulses automatically, without requiring any further attention from the processor. While the timer quietly keeps track of time, the CPU is free to execute the rest of the program.

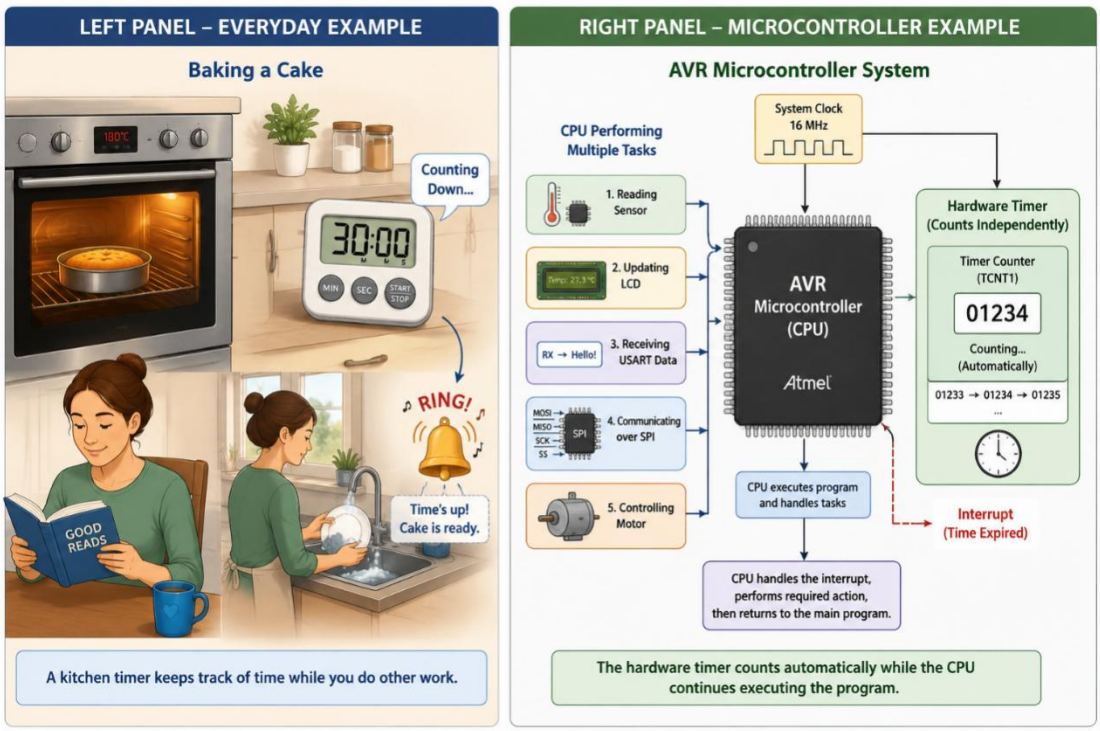
When a specified amount of time has elapsed, the timer can notify the CPU by setting a status flag or generating an interrupt. The CPU briefly pauses its current task, handles the timed event, and then continues exactly where it left off.

In many ways, a hardware timer behaves just like the kitchen timer on your countertop—it counts in the background and lets you know when the waiting is over.

Throughout this chapter, you will discover that timers are far more than simple clocks. They can measure time intervals with remarkable precision, generate accurate delays, produce periodic events, create PWM signals for controlling motors and LEDs, measure external signals, and even keep an entire embedded application running on schedule.

In short, **timers are the microcontroller's built-in clocks, stopwatches, alarm systems, and timekeepers**, working silently in the background so that the CPU can focus on solving the real problem.

Illustration: *Why a Microcontroller Needs a Timer*



Meet the Hardware Timer

Before we learn how to configure a timer, let us first understand what it actually is.

Despite its impressive name, a **hardware timer** is nothing more than a **binary counter**. Its entire job is surprisingly simple—it counts clock pulses, one after another, without ever becoming tired or distracted.

Imagine standing beside a road and counting passing cars.

Each time a car passes, you increase your count by one.

One...

Two...

Three...

Four...

The timer behaves in exactly the same way. Instead of counting cars, however, it counts the electrical clock pulses generated by the microcontroller's system clock.

Every time a clock pulse arrives, the timer increases its value by **one**. This counting happens continuously as long as the timer is running. Whether the CPU is executing instructions, waiting for serial data, blinking LEDs, or performing mathematical calculations, the timer quietly continues counting in the background.

One of the most fascinating aspects of AVR microcontrollers is that **the CPU never increments the timer itself**.

The timer contains dedicated digital hardware whose only purpose is to count clock pulses. Once the timer is enabled, the counting process becomes completely automatic. The CPU simply configures the timer, starts it, and then moves on to other tasks. The timer continues counting independently until something important happens, such as reaching a compare value or overflowing.

This is one of the first moments where many students begin to appreciate the elegance of embedded hardware.

Rather than asking the processor to perform every task in software, the AVR delegates repetitive work to specialized hardware peripherals. The timer becomes a tiny assistant that works tirelessly in the background, freeing the CPU to focus on the main application.

Timer0 — The 8-bit Timer

The ATmega328P contains an **8-bit timer** called **Timer0**. An 8-bit binary number can represent **256 different values**, ranging from **0** to **255**.

As clock pulses arrive, Timer0 counts like this:

```
0 → 1 → 2 → 3 → 4 → ... → 253 → 254 → 255
```

After reaching **255**, it cannot count any higher. On the very next clock pulse, it wraps back to **0** and begins counting again. This event is known as an **overflow**, which we will study later in this chapter.

Timer1 — The 16-bit Timer

The microcontroller also provides a larger timer called **Timer1**. Unlike Timer0, it uses **16 bits**, allowing it to store much larger numbers.

Instead of counting only to **255**, Timer1 counts all the way to **65,535** before wrapping back to zero.

```
0 → 1 → 2 → 3 → ... → 65,533 → 65,534 → 65,535
```

Because it has a much larger counting range, Timer1 is ideal for measuring longer time intervals or generating lower-frequency timing events without overflowing as quickly.

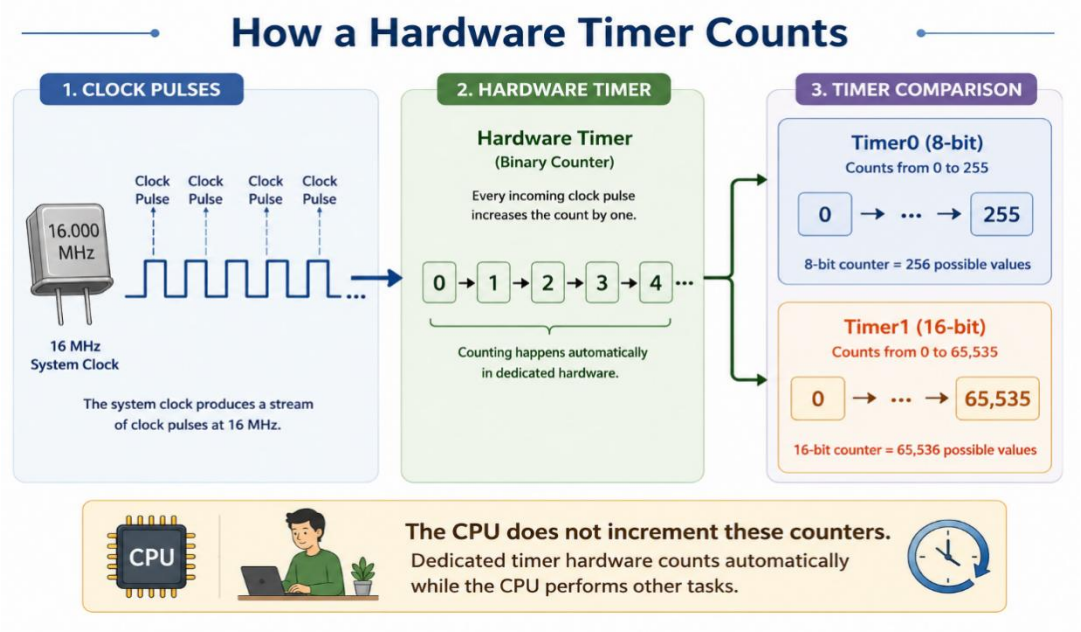
Two Timers, Two Different Jobs

Although both timers perform exactly the same basic task, they differ in how much they can count before starting over.

Timer	Size	Counting Range	Maximum Count
Timer0	8-bit	0 → 255	255
Timer1	16-bit	0 → 65,535	65,535

A larger timer does not count faster—it simply counts **farther** before overflowing.

As you continue through this chapter, you will discover that these simple counters are capable of much more than merely keeping track of numbers. By combining them with prescaler, compare registers, and interrupts, the AVR timer transforms into one of the most powerful peripherals inside the microcontroller.



How a Hardware Timer Counts?

The illustration on the previous page shows the complete journey of a clock pulse as it is transformed into an accurate measurement of time. Although the diagram appears simple, it represents one of the most important hardware peripherals inside every AVR microcontroller.

Let us explore each section in detail.

The System Clock — The Heartbeat of the Microcontroller

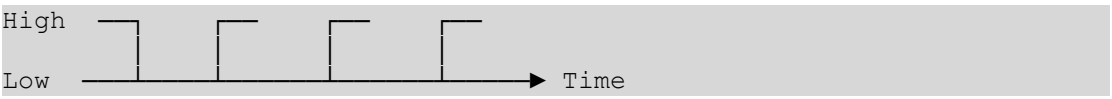
Every digital system needs a source of timing. In an AVR microcontroller, this timing is provided by the **system clock**, which is usually generated by a crystal oscillator.

For an Arduino Uno or an ATmega328P, the clock typically operates at **16 MHz**, meaning it produces:

16,000,000 clock pulses every second.

You can think of these pulses as the heartbeat of the microcontroller. Every instruction executed by the CPU, every peripheral, and every timer operates in synchronization with this clock.

The clock produces an endless stream of electrical pulses that look like this:



Each rising edge (or internal clock event) represents one opportunity for the timer to increase its count.

Without the system clock, the timer would have nothing to count.

Every Clock Pulse Increases the Counter

The hardware timer receives these clock pulses one after another.

Its job is wonderfully simple. Whenever a clock pulse arrives, the timer adds **one** to its internal counter.

```
Clock Pulse #1   → Counter = 0
Clock Pulse #2   → Counter = 1
Clock Pulse #3   → Counter = 2
Clock Pulse #4   → Counter = 3
Clock Pulse #5   → Counter = 4
...
```

This process continues continuously while the timer is running.

Notice something remarkable.

The timer does **not** need instructions from the CPU to perform this counting. It simply reacts to each incoming clock pulse automatically.

This is dedicated hardware performing a repetitive task far more efficiently than software ever could.

The Timer Is a Binary Counter

Internally, the timer stores its value in a binary register. Although programmers usually read the count as decimal numbers such as 25, 100, or 250, the hardware actually stores these values in binary.

For example,

```
Decimal      Binary
0        =   00000000
1        =   00000001
2        =   00000010
3        =   00000011
4        =   00000100
```

Each incoming clock pulse causes the binary number to increase by one. This process happens millions of times every second without any software intervention.

Timer0 — The 8-Bit Counter

The first timer inside the ATmega328P is **Timer0**. Because it is an **8-bit** counter, it contains eight storage bits. Eight bits can represent exactly **256 different values**. Therefore Timer0 counts like this:

```
0
↓
1
↓
2
↓
3
↓
...
↓
253
↓
254
↓
255
```

Once the counter reaches **255**, it has no larger value available. On the next clock pulse it returns to **0** and starts counting again. This event is called an **overflow**, which we shall study in detail later.

Although Timer0 has a relatively small counting range, it is extremely useful for generating short delays, periodic interrupts, and timing events that occur frequently.

Timer1 — The 16-Bit Counter

The ATmega328P also contains a much larger timer called **Timer1**. Instead of eight bits, it contains **sixteen bits**. A 16-bit binary number can represent:

65,536 different values

which means Timer1 counts from

```
0
↓
1
↓
2
↓
...
↓
65,533
↓
65,534
↓
65,535
```

Only after reaching **65,535** does the timer wrap back to zero.

Because it can count much farther before overflowing, Timer1 is ideal whenever longer timing intervals or greater timing precision are required.

This is why many advanced embedded applications use Timer1 for accurate clocks, servo control, pulse measurements, and high-resolution timing.

Bigger Does Not Mean Faster

Students often assume that Timer1 must count faster because it is larger. In reality, both timers receive exactly the same clock source. The difference is not **speed** but **capacity**.

Think of two measuring cups. One holds **250 mL**, while the other holds **2 litres**. Both can be filled at exactly the same rate. The larger cup simply takes longer before becoming full.

Timers work in precisely the same way. Timer1 does not count faster than Timer0—it simply continues counting much longer before overflowing.

The Hidden Assistant Inside the Microcontroller

Perhaps the most fascinating part of this entire process is that the CPU is not involved in counting at all. After configuring the timer, the CPU simply tells it,

"Start counting."

From that moment onward, dedicated digital hardware takes over.

While the CPU is busy reading sensors, communicating over USART, updating displays, or performing calculations, the timer quietly continues counting every clock pulse with perfect accuracy. The CPU may never even look at the timer until an important event occurs.

This division of labour is one of the reasons microcontrollers are so efficient. Instead of asking the CPU to perform every repetitive task in software, the AVR assigns specialized jobs to dedicated hardware peripherals.

The timer is one such peripheral—a silent assistant that never sleeps, never misses a pulse, and never loses count.

Key Points

- A hardware timer is essentially a **binary counter**.
- Every incoming clock pulse increases the counter by **one**.
- **Timer0** is an **8-bit** counter that counts from **0 to 255**.
- **Timer1** is a **16-bit** counter that counts from **0 to 65,535**.
- Timer1 does **not** count faster than Timer0—it simply counts **farther** before overflowing.
- Once started, the timer counts **automatically in hardware**.
- The CPU does **not** increment the timer value; it is free to perform other tasks while the timer operates independently.

The Timer Clock and Prescaler

In the previous section, we learned that a hardware timer is simply a counter. Every clock pulse arriving at the timer causes the counter to increase by one. That sounds simple enough—but it also raises an important question.

How fast does the timer count?

The answer depends entirely on the clock that drives it. Most ATmega328P-based development boards, including the Arduino Uno and the MikroDuino platform, operate from a **16 MHz system clock**.

This means the oscillator generates:

16,000,000 clock pulses every second.

If the timer were connected directly to this clock, it would receive every one of those pulses. Each pulse would increase the timer by one. In other words, the timer would count:

16 million times every second.

That is astonishingly fast. To appreciate just how fast this is, let us consider the smallest timer inside the AVR—the 8-bit **Timer0**.

Since Timer0 can only count from **0 to 255**, it needs just **256 clock pulses** before it overflows. At a clock frequency of **16 MHz**, those 256 pulses arrive in only:

That is only **16 microseconds**.

To put this into perspective, a human eye blinks in roughly **300 milliseconds**. During that single blink, Timer0 would overflow nearly **19,000 times**!

Clearly, counting at full clock speed is far too fast for many practical applications. Suppose you want an LED to blink once every second. Or you wish to read a temperature sensor every 100 milliseconds. Or perhaps you need to generate a one-second delay. If the timer overflows every 16 microseconds, it becomes difficult to produce these much longer time intervals.

Fortunately, the AVR provides a remarkably elegant solution. It is called the **prescaler**.

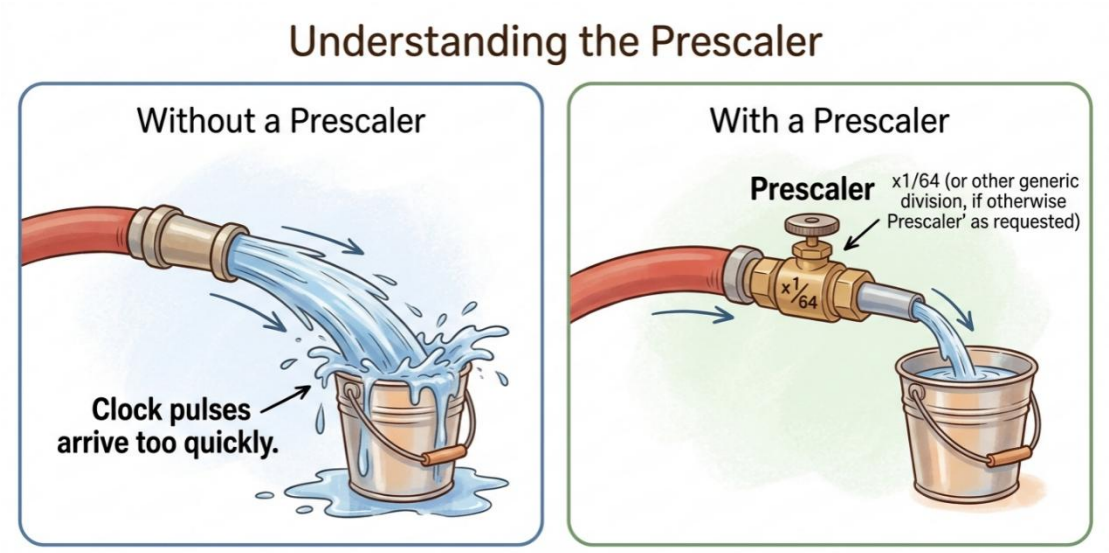
Slowing Down the Clock

Imagine that you need to fill a bucket with water. You connect the bucket directly to a large fire hose. Water rushes into the bucket with tremendous force. Within a fraction of a second, the bucket overflows.

The problem is not that the bucket is too small. The problem is that the water is arriving far too quickly. Now imagine placing a control valve between the fire hose and the bucket. By partially closing the valve, you reduce the flow of water. The bucket now fills much more slowly, giving you complete control over the filling process.

The prescaler works in exactly the same way. Instead of reducing the flow of water, it reduces the flow of **clock pulses**. Rather than allowing every clock pulse to reach the timer, the prescaler allows only selected pulses to pass.

The timer therefore counts more slowly, making it possible to measure much longer periods of time.



The prescaler reduces the flow of clock pulses, allowing the timer to count more slowly.

How the Prescaler Works

Inside the microcontroller, the prescaler sits between the **system clock** and the **timer counter**. Instead of forwarding every clock pulse, it divides the incoming clock frequency before sending it to the timer.

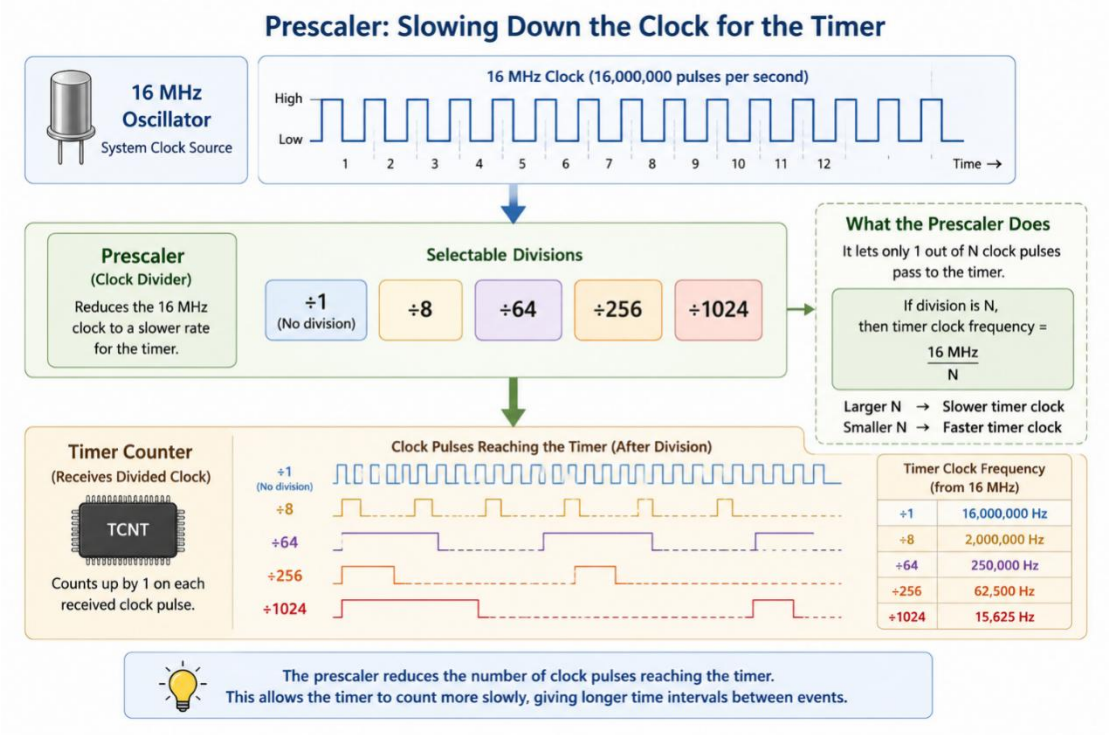
For example, suppose the prescaler is configured to divide the clock by **8**.

The system clock still produces **16 million pulses every second**, but the timer receives only:

[16,000,000/8]

2,000,000 pulses per second

The timer now counts eight times more slowly than before. If the prescaler divides by **64**, the timer receives only one out of every sixty-four clock pulses. The counting becomes slower still. This simple idea dramatically increases the amount of time the timer can measure before overflowing.



Available Prescaler Values

The AVR timers allow several selectable clock divisions.

Each division changes how quickly the timer counts.

Prescaler	Timer Clock
÷1	16 MHz
÷8	2 MHz
÷64	250 kHz
÷256	62.5 kHz
÷1024	15.625 kHz

A smaller division produces a faster timer. A larger division produces a slower timer.

Choosing the correct prescaler is one of the most important decisions when configuring a timer because it determines the timer's resolution and maximum measurable interval.

One Pulse Out of Many

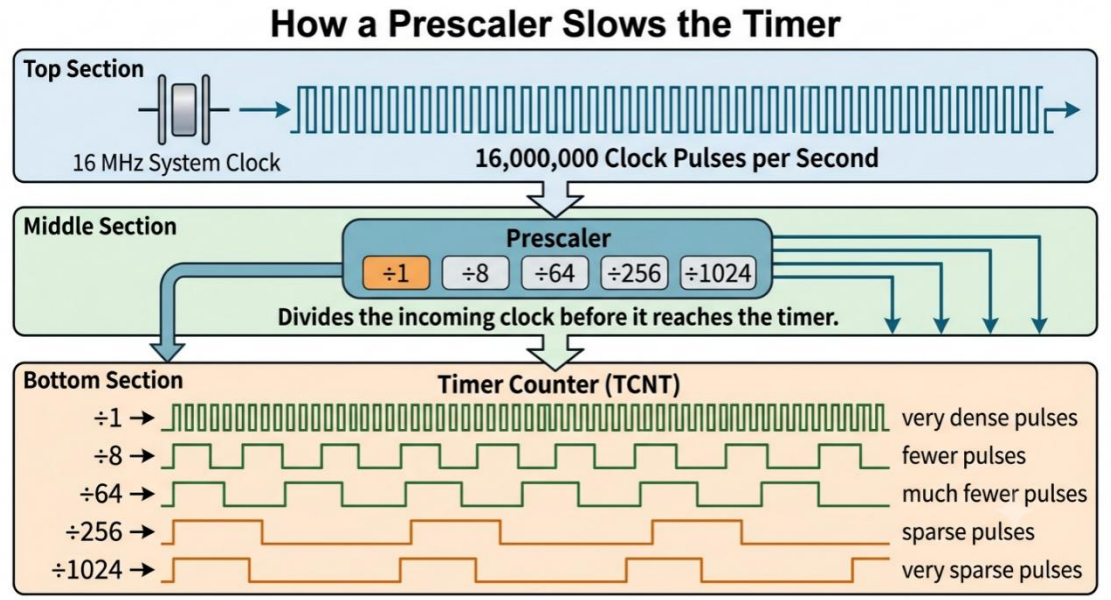
Another way to visualize the prescaler is to imagine a security guard standing at a gate. Thousands of people are trying to enter.

The guard decides to allow only:

- every pulse through (÷1),

- every 8th pulse (÷8),
- every 64th pulse (÷64),
- every 256th pulse (÷256),
- or every 1024th pulse (÷1024).

The timer only sees the pulses that the guard allows through. As fewer pulses arrive, the timer counts more slowly. Nothing about the timer itself has changed. Only the rate at which it receives clock pulses has changed.



Choosing the Right Prescaler

There is no single "best" prescaler. The correct choice depends entirely on the application. If you need extremely precise timing over very short intervals, you would choose a small division such as **÷1** or **÷8**, allowing the timer to count rapidly.

If you need to measure longer intervals or generate slower periodic events, you would choose a larger division such as **÷256** or **÷1024**, allowing the timer to count much more slowly before overflowing.

Selecting the appropriate prescaler is much like choosing the correct gear on a bicycle. A low gear provides quick response but covers less distance with each pedal rotation, while a high gear moves farther with each turn but responds more gradually. Similarly, a small prescaler gives high timing resolution, whereas a larger prescaler extends the timer's usable range.

AVR Timer Operating Modes (ATmega328P)

Up to this point, we have learned that a timer is essentially a counter driven by the system clock or an external clock source, and that the timer speed can be adjusted using a prescaler. The next question naturally arises:

What should the timer do while it is counting?

The answer depends on the **operating mode** selected for the timer.

A timer can be configured to perform different tasks depending on the application. Sometimes we simply want it to count elapsed time. Sometimes we want it to generate an interrupt after a specific interval. In other applications we may want it to generate digital waveforms or control motor speed.

Instead of designing separate hardware for each task, AVR timers provide several **operating modes**, allowing the same hardware timer to perform many different functions.

The ATmega328P timers support four fundamental operating modes:

- Normal Mode
- Clear Timer on Compare Match (CTC) Mode
- Fast PWM Mode
- Phase Correct PWM Mode

Each mode changes the way the timer counts and what happens when the timer reaches certain values.

Choosing the Right Timer Mode

The operating mode determines three important characteristics of the timer:

- How the timer counts
- When the counter resets
- What event occurs when the count reaches a specific value

Different applications require different behavior.

Timer Mode	Primary Purpose
Normal Mode	Simple counting and overflow timing
CTC Mode	Precise periodic timing
Fast PWM	Waveform generation (high-speed PWM)
Phase Correct PWM	Symmetrical waveform generation

For now, our focus will remain on **Normal Mode** and **CTC Mode**, which are the two modes most commonly used for creating delays, timers, periodic interrupts, and event scheduling.

The PWM modes will be introduced briefly here and studied in detail later in this chapter.

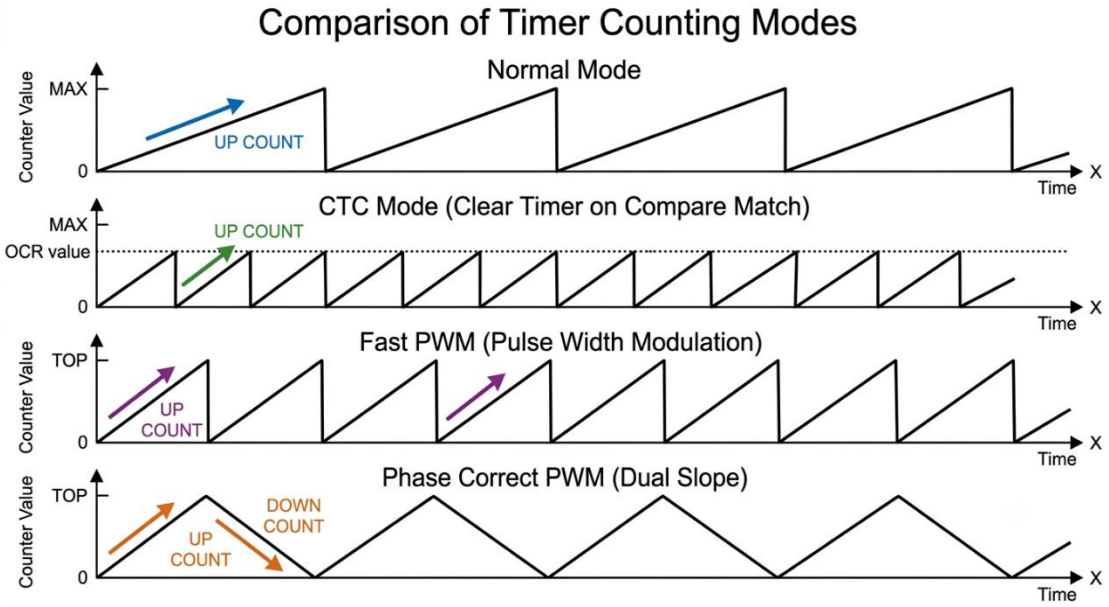
How Timer Modes Affect Counting

Although every timer contains a counter register (TCNTn), the way this counter behaves changes depending on the selected mode.

The timer may

- count continuously until it overflows,
- reset automatically at a compare value,
- count upward only,
- count upward and downward.

These different counting methods give rise to the four operating modes.



Normal Mode

Normal Mode is the simplest operating mode available. In this mode the timer behaves exactly like a basic binary counter. After being started, the timer increments by one on every timer clock.

For an 8-bit timer, the counting sequence is

```
0
1
2
3
...
```

```
253
254
255
0
1
2
...
```

When the maximum value is reached, the next clock pulse causes the timer to **overflow**, returning the counter to zero.

This overflow is one of the most useful events produced by the timer.

Counting Behavior

The timer performs only one task:

- Count upward.
- Overflow.
- Continue counting.

There are no automatic resets before overflow. No comparison value affects the counter. The timer simply keeps running.

Overflow Event

When the counter changes from its maximum value back to zero, an **overflow** occurs.

For example,

An 8-bit timer counts

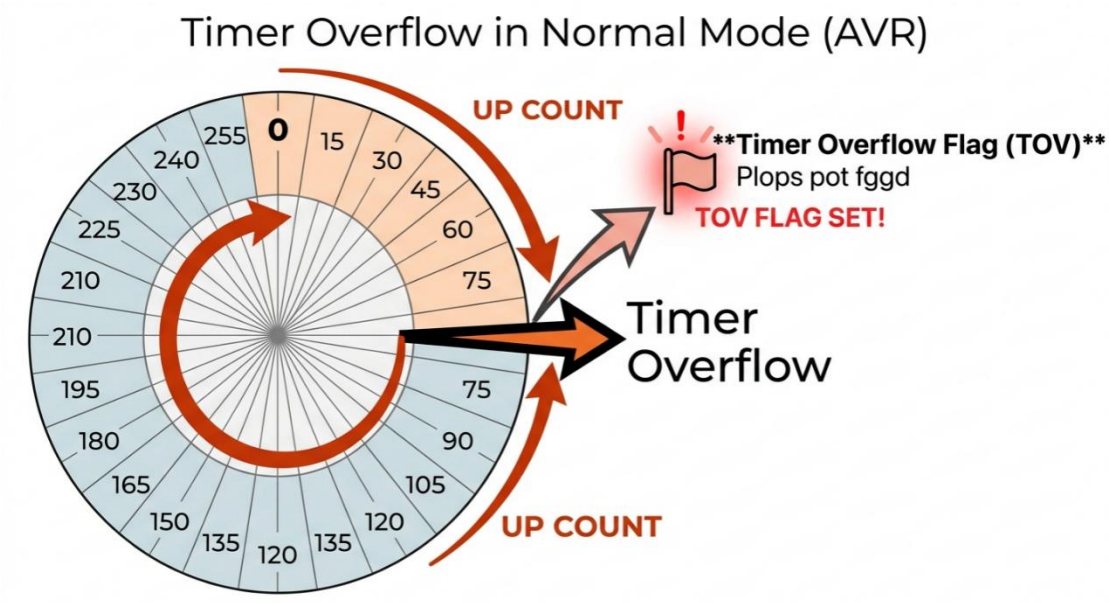
```
254
255
0   ← Overflow
1
```

A 16-bit timer counts

```
65534
65535
0   ← Overflow
1
```

The overflow can

- set the Timer Overflow Flag,
- generate an interrupt (if enabled),
- notify software that a specific time interval has elapsed.



Advantages of Normal Mode

Normal Mode is simple and efficient for

- creating software timers,
- measuring elapsed time,
- periodic interrupts,
- event counting,
- frequency measurement.

Limitations

The overflow period is fixed by

- timer size,
- clock frequency,
- prescaler.

If a different timing interval is needed, software often has to count multiple overflows. This makes Normal Mode slightly less convenient when an exact timing interval is required.

CTC Mode (Clear Timer on Compare Match)

CTC stands for **Clear Timer on Compare Match.** This is one of the most useful timer modes available in the ATmega328P. Instead of allowing the timer to count until overflow, CTC mode allows us to choose our own maximum count.

Basic Idea

A second register called the **Output Compare Register (OCRnA or OCRnB)** stores a comparison value.

While counting, the timer continuously compares

```
TCNTn
```

with

```
OCRn
```

When both values become equal, a **Compare Match** occurs. Immediately afterwards, the timer resets back to zero and begins counting again.

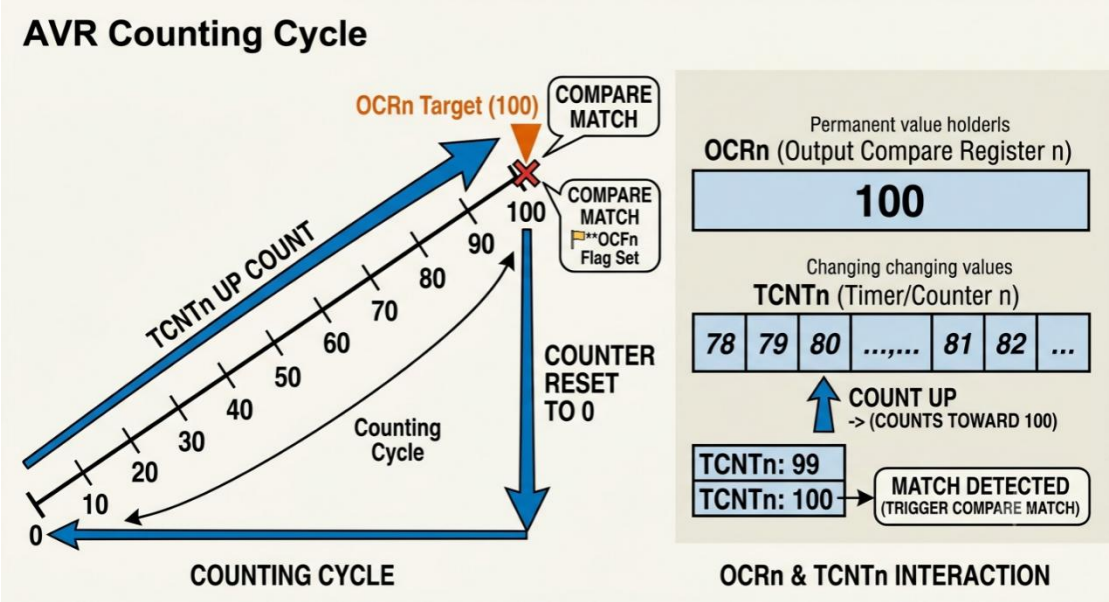
Instead of counting

```
0
1
2
...
255
0
```

the timer may count

```
0
1
2
...
99
100 ← Compare Match
0
1
2
...
```

Here, 100 becomes the new counting limit.



Compare Match Event

Whenever

```
TCNTn == OCRn
```

the hardware performs several possible actions.

It can

- reset the timer,
- set the Compare Match Flag,
- generate an interrupt,
- optionally toggle an output pin.

Which actions occur depends on the configuration registers.

Why CTC Mode Is Useful

Suppose an application requires an interrupt every

```
1 millisecond
```

Using Normal Mode, the programmer may have to wait for overflow and count several overflows. Using CTC Mode, the compare value can be chosen so that the timer resets exactly every 1 ms. No software overflow counting is required. This makes timing more accurate and simplifies the program.

Typical Applications

CTC Mode is widely used for

- periodic interrupts,
- real-time clocks,
- task scheduling,
- baud rate generation,
- square wave generation,
- timing delays.

Fast PWM Mode

The ATmega328P timers are also capable of generating digital waveforms automatically. One of the most commonly used waveform generation modes is

Fast PWM Mode.

PWM stands for **Pulse Width Modulation.**

Instead of simply counting time, the timer automatically generates a repetitive digital waveform whose pulse width can be precisely controlled.

In Fast PWM mode,

the timer counts

```
0
1
2
...
TOP
0
1
2
...
```

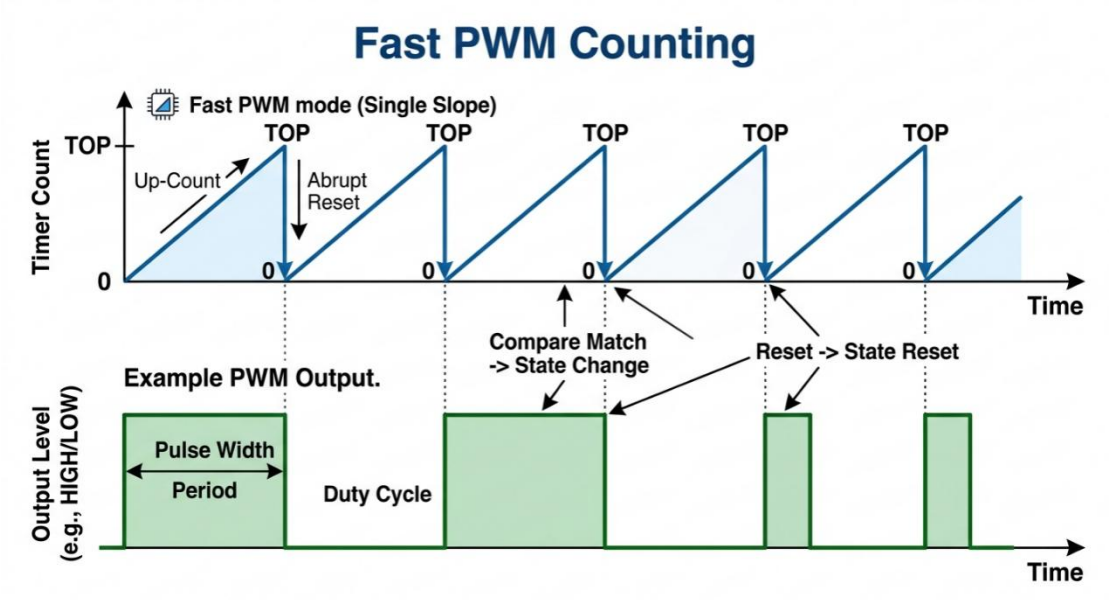
The timer repeatedly counts upward and immediately returns to zero.

This produces a high-frequency PWM signal. The exact relationship between timer counts and output waveform will be studied in detail later in this chapter.

Typical Applications

Fast PWM is commonly used for

- LED brightness control,
- motor speed control,
- power regulation,
- audio generation,
- switching power supplies.



Phase Correct PWM Mode

Another waveform generation mode is

Phase Correct PWM Mode.

Unlike Fast PWM, the timer does not simply count upward.

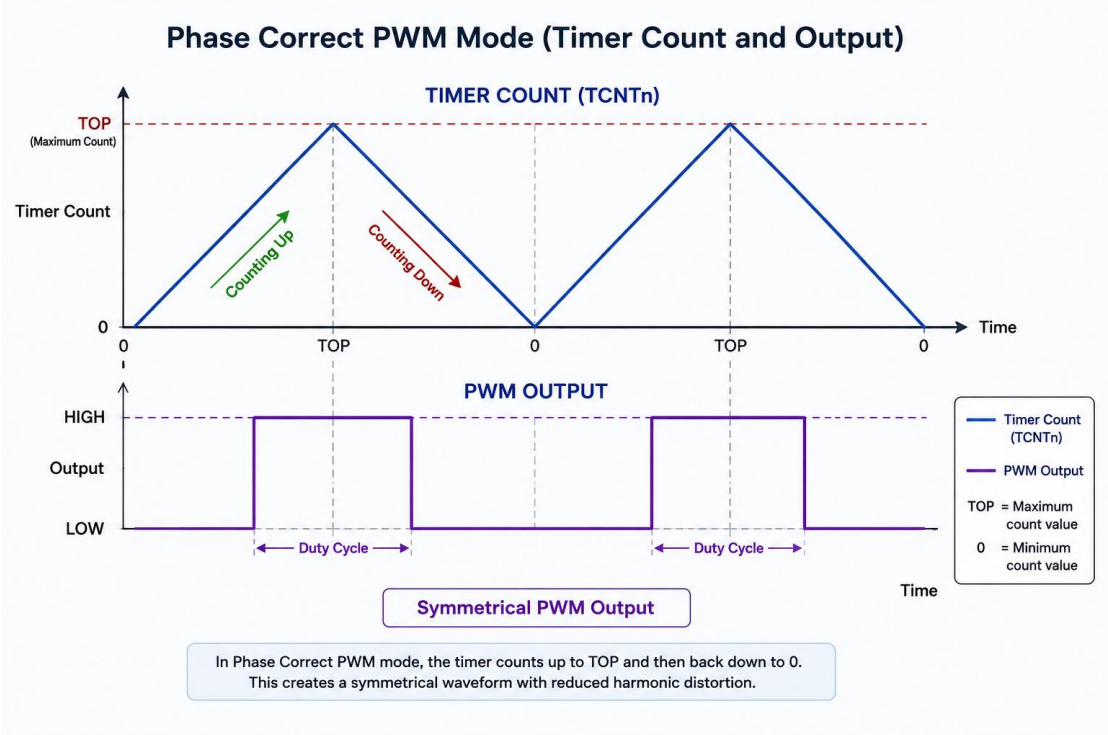
Instead, it counts

```
0
1
2
...
TOP
TOP-1
...
2
1
0
```

The counter repeatedly moves upward and then downward.

Because the counting is symmetrical, the generated waveform is also symmetrical. This reduces timing asymmetry and is particularly useful in applications where waveform quality is important.

The detailed operation of Phase Correct PWM, including duty cycle calculations and output behavior, will be covered later in this chapter.



Comparing the Four Timer Modes

Each timer mode changes the counting behavior to suit different applications. Understanding these differences helps in selecting the most appropriate mode for a given task.

Mode	Counting Behaviour	Reset Condition	Primary Use
Normal Mode	Counts upward continuously	Overflow (maximum count)	Time measurement, overflow interrupts
CTC Mode	Counts upward to a programmable compare value	Compare Match	Precise periodic timing, scheduled interrupts
Fast PWM	Counts upward repeatedly to TOP	Automatic reset at TOP	High-speed waveform generation
Phase Correct PWM	Counts upward to TOP, then downward to 0	Automatic direction reversal	Symmetrical waveform generation

Summary

The ATmega328P timer hardware supports four primary operating modes, each tailored to a different class of applications. **Normal Mode** provides straightforward counting until overflow, making it suitable for basic timing and elapsed-time measurement. **CTC Mode** improves precision by resetting the timer at a programmable compare value, making it ideal for periodic interrupts and accurate time intervals. **Fast PWM** and **Phase Correct PWM** extend the timer beyond simple counting into hardware waveform generation. Although these PWM modes are introduced here to complete the overview of timer operation, their configuration,

timing relationships, and practical applications will be explored in detail in the dedicated PWM section later in this chapter.